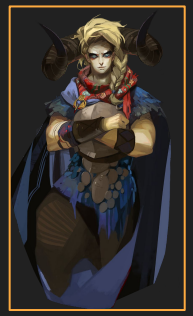
IT UNIVERSITY OF COPENHAGEN

Game Programming 25

L02: Collisions

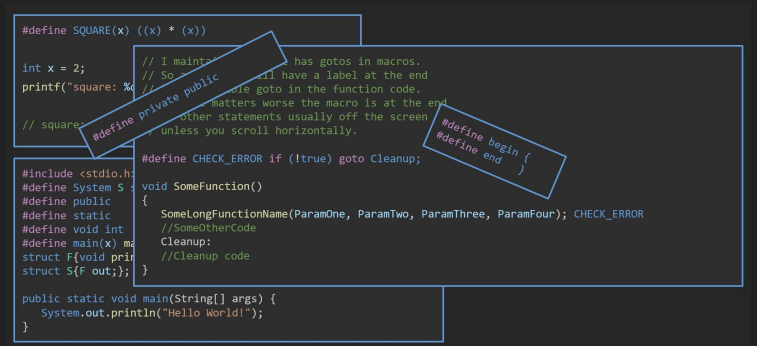
Chris Carvelli

Recap



images and textures

sprites



preprocessor directives and macros

C++ memory layout

Exercise 01 review - Add projectiles (init)

```
// asteroids init
{
    for(int i = 0; i < NUM_ASTEROIDS; ++i)
    {
        Entity* entity = &game_state->asteroids[i];

        entity->size = asteroid_collision_radius;
        entity->rect.w = entity_size_world;
        entity->rect.h = entity_size_world;
        entity->texture_atlas = game_state->texture_atlas;
        entity->texture_rect.w = entity_size_texture;
        entity->texture_rect.h = entity_size_texture;
        entity->texture_rect.x = entity_size_texture * asteroid_sprite_x;
        entity->texture_rect.y = entity_size_texture * asteroid_sprite_y;
    }
}
```

```
// projectiles init
{
    for(int i = 0; i < NUM_PROJECTILES; ++i)
    {
        Entity* entity = &game_state->projectiles[i];

        entity->size = projectile_collision_radius;
        entity->rect.w = entity_size_world;
        entity->rect.h = entity_size_world;
        entity->texture_atlas = game_state->texture_atlas;
        entity->texture_rect.w = entity_size_texture;
        entity->texture_rect.h = entity_size_texture;
        entity->texture_rect.x = entity_size_texture * projectile_sprite_x;
        entity->texture_rect.y = entity_size_texture * projectile_sprite_y;
    }
}
```

literally the same code
different data

Exercise 01 review - Add projectiles (update)

```
// asteroids update

{ /* spawning logic */ }

for(int i = 0; i < NUM_ASTEROIDS; ++i)
{
    Entity* e = &game_state->asteroids[i];
    if(!e->alive)
        continue;

    e->position.y += context->delta * e->velocity;

    if(entity_collision_check(e, &game_state->player))
    {
        //SDL_Log("game over");
        game_state->game_over = true;
        return;
    }

    // despawn when out of bounds
    if(e->position.y > context->window_h + e->texture_rect.h)
        e->alive = false;
}
```

```
// projectiles update
for(int i = 0; i < NUM_PROJECTILES; ++i)
{
    Entity* e = &game_state->projectiles[i];
    if(!e->alive)
        continue;

    e->position.y -= context->delta * e->velocity;

    // despawn when out of bounds
    if(e->position.y < -e->texture_rect.h)
        e->alive = false;

    // despawn projectile and asteroids if they overlap
    for(int i = 0; i < NUM_ASTEROIDS; ++i)
    {
        Entity* asteroid_curr = &game_state->asteroids[i];
        if(!asteroid_curr->alive)
            continue;

        if(entity_collision_check(e, asteroid_curr))
        {
            e->alive = false;
            asteroid_curr->alive = false;
        }
    }
}
```

lots of similarities

Exercise 01 review - Beware shadowing!

ops, this is wrong!

but why does it work anyway here?

```
// projectiles update
for(int i = 0; i < NUM_PROJECTILES; ++i)
{
    Entity* e = &game_state->projectiles[i];
    if(!e->alive)
        continue;

    e->position.y -= context->delta * e->velocity;

    // despawn when out of bounds
    if(e->position.y < -e->texture_rect.h)
        e->alive = false;

    // despawn projectile and asteroids if they overlap
    for(int i = 0; i < NUM_ASTEROIDS; ++i)
    {
        Entity* asteroid_curr = &game_state->asteroids[i];
        if(!asteroid_curr->alive)
            continue;

        if(entity_collision_check(e, asteroid_curr))
        {
            e->alive = false;
            asteroid_curr->alive = false;
        }
    }
}
```

Exercise 01 review - Add projectiles (render)

```
// asteroids render
for(int i = 0; i < NUM_ASTEROIDS; ++i)
{
    Entity* asteroid_curr = &game_state->asteroids[i];
    if(!asteroid_curr->alive)
        continue;

    entity_render(context, asteroid_curr);
}
```

```
// projectiles render
for(int i = 0; i < NUM_PROJECTILES; ++i)
{
    Entity* entity = &game_state->projectiles[i];
    if(!entity->alive)
        continue;

    entity_render(context, entity);
}
```

literally the same code

Exercise 01 review - Spaw entities

```
// spawn a new entity
// NOTE: this works even if we are passing an array (ie, the original declaration of `GameState::asteroids` and `GameState::projectiles`
//       as `Entity asteroids[NUM_ASTEROIDS]), since they are *mostly* equivalent.
Entity* entity_spawn(Entity* pool, int max_amount, float x, float y, float velocity)
{
    // find id of first alive entity (checking we are not skipping past the end of the array
    int new_entity_idx = 0;
    while(new_entity_idx < max_amount && pool[new_entity_idx].alive)
        ++new_entity_idx;

    if(new_entity_idx == max_amount)
    {
        // NOTE: since we changed `GameState::asteroids` and `GameState::projectiles` from array of entities to pointers,
        //       in theory we could resize entity array here. It would require some additional changes to keep track of
        //       the new `max_amount` of entities in the pool (and we would need to set a ceiling anyway, we can't keep
        //       spawning new entities forever
        SDL_Log("[WARNING] too many entities, cannot spawn more!");
        return NULL;
    }

    Entity* entity = &pool[new_entity_idx];

    entity->alive = true;
    entity->position.x = x;
    entity->position.y = y;
    entity->velocity   = velocity;

    // return the newly spawned entity, so we can do additional initialization if needed
    return entity;
}
```

Exercise 01 review - Reset game state

can be done better

```
// beginning of init()

// reset game state
{
    // NOTE: from `SDL_Free` documentation: "if `mem` [the parameter] is NULL, this function does nothing
    //       ie, we don't need to test for null here (like most implementations of the standard `free`).
    SDL_free(game_state->asteroids);
    SDL_free(game_state->projectiles);

    // NOTE: from `SDL_DestroyTexture`: "Passing NULL or an otherwise invalid texture will set the SDL error message to "Invalid texture""
    //       ie, we COULD avoid this test, but if you're keeping track of SDL internal messages your logs will be full "errors" that will drown
    //       the real problems in a sea of noise, so let's avoid it
    if(game_state->texture_atlas)
        SDL_DestroyTexture(game_state->texture_atlas);
}
// zero out memory (need to do this explicitly if we want to reset the game state)
SDL_memset(game_state, 0, sizeof(GameState));

{ /* load textures */ }

// game state
{
    game_state->asteroids = (Entity*)SDL_calloc(NUM_ASTEROIDS, sizeof(Entity));
    game_state->projectiles = (Entity*)SDL_calloc(NUM_PROJECTILES, sizeof(Entity));
    game_state->asteroid_spawning_period = asteroid_spawning_period_base;
}

// entities below...
```

```
// beginning of main()
init(context, game_state);

// beginning of update()
if (game_state->game_over)
    init(context, game_state);
```

Exercise 01 review - Reset game state, improved

```
static void init(SDLContext* context, GameState* game_state)
{
    // zero out memory
    SDL_memset(game_state, 0, sizeof(GameState));

    { /* load textures */ }

    game_state->asteroids = (Entity*)SDL_calloc(NUM_ASTEROIDS, sizeof(Entity));
    game_state->projectiles = (Entity*)SDL_calloc(NUM_PROJECTILES, sizeof(Entity));

    // this has to be reset every time we reset the game
    // game_state->asteroid_spawning_period = asteroid_spawning_period_base;


    reset(context, game_state);
}
```

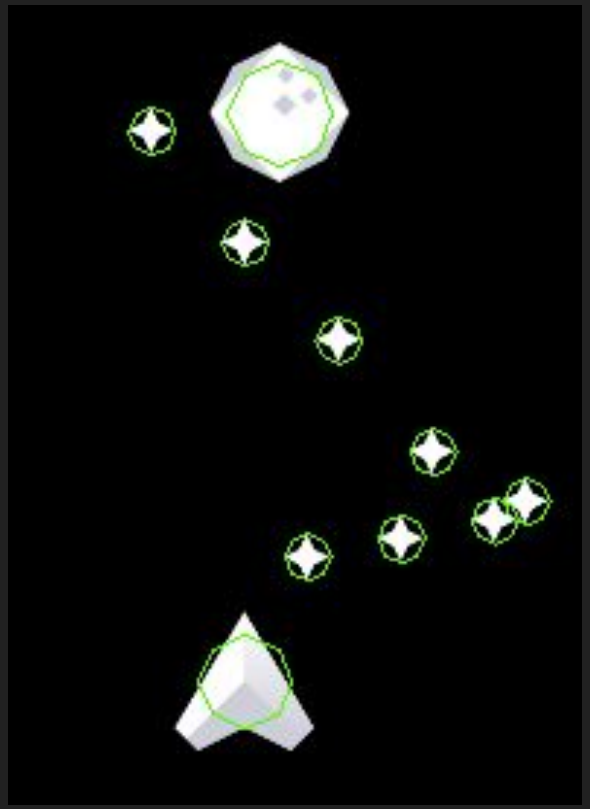
```
// beginning of main()
init(context, game_state);

// beginning of update()
if (game_state->game_over)
    reset(context, game_state);
```

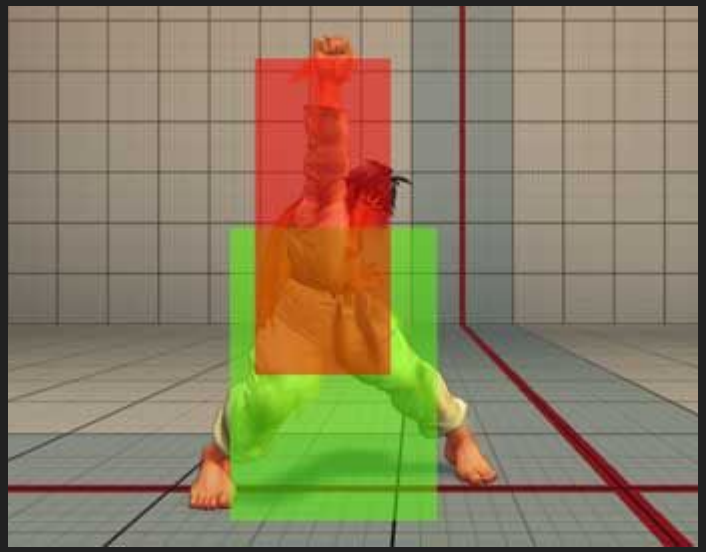
Exercise 01 review - handling projectile collisions

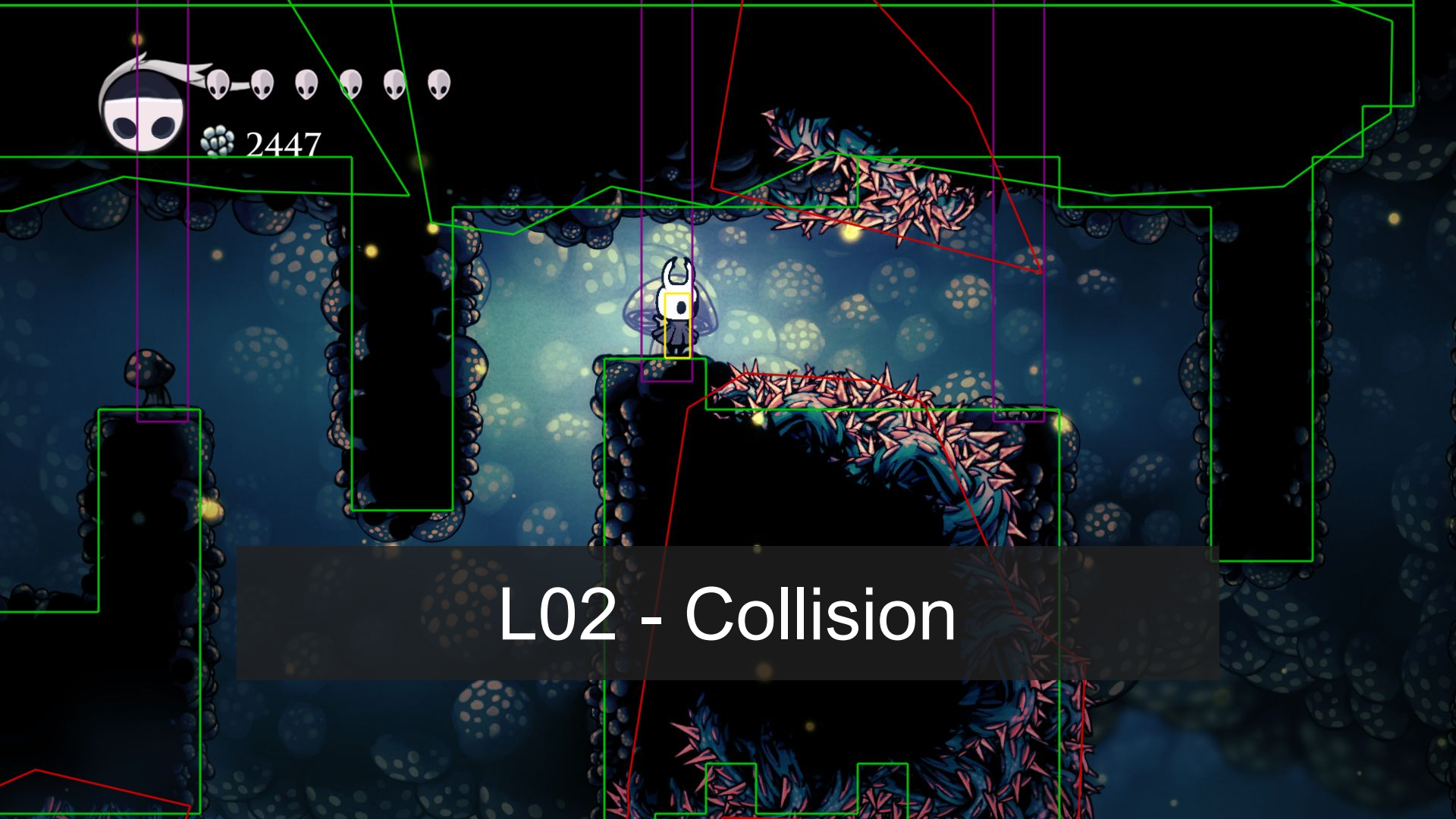
```
// projectile update
for(int i = 0; i < NUM_PROJECTILES; ++i)
{
    Entity* entity = &game_state->projectiles[i];
    if(!entity->alive)
        continue;

    // other update stuff...

    // actual collision detection
    for(int j = 0; j < NUM_ASTEROIDS; ++j)
    {
        Entity* asteroid_curr = &game_state->asteroids[j];
        if(!asteroid_curr->alive)
            continue;

        if(entity_collision_check(entity, asteroid_curr))
        {
            entity->alive = false;
            asteroid_curr->alive = false;
        }
    }
}
```

today’s main topic!

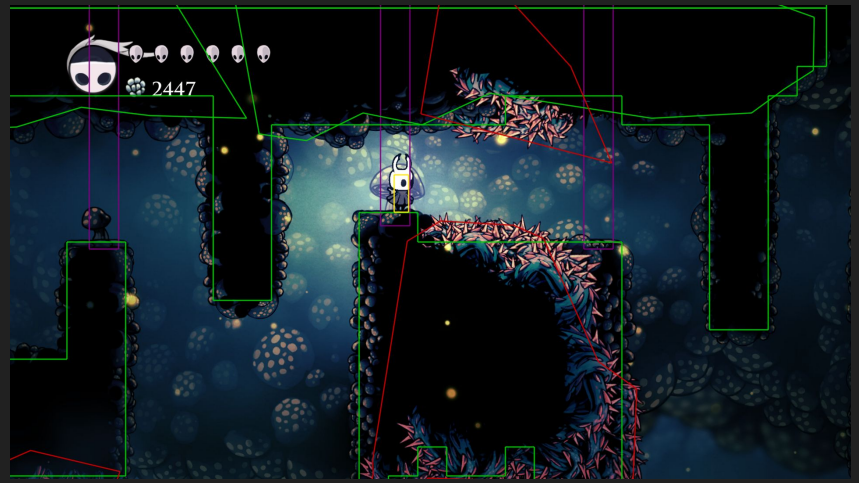
Exercise 01 review - collision size





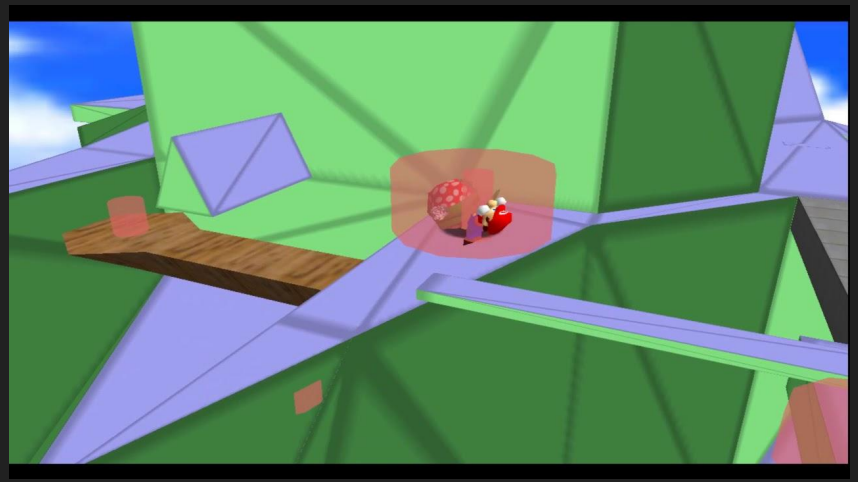

2447
L02 - Collision

Problem statement: How do we…



- …efficiently detect collisions between arbitrary shapes?
- …tackle the quadratic nature of checking multiple colliders we saw in exercise 01?

- …separate entities that are colliding?



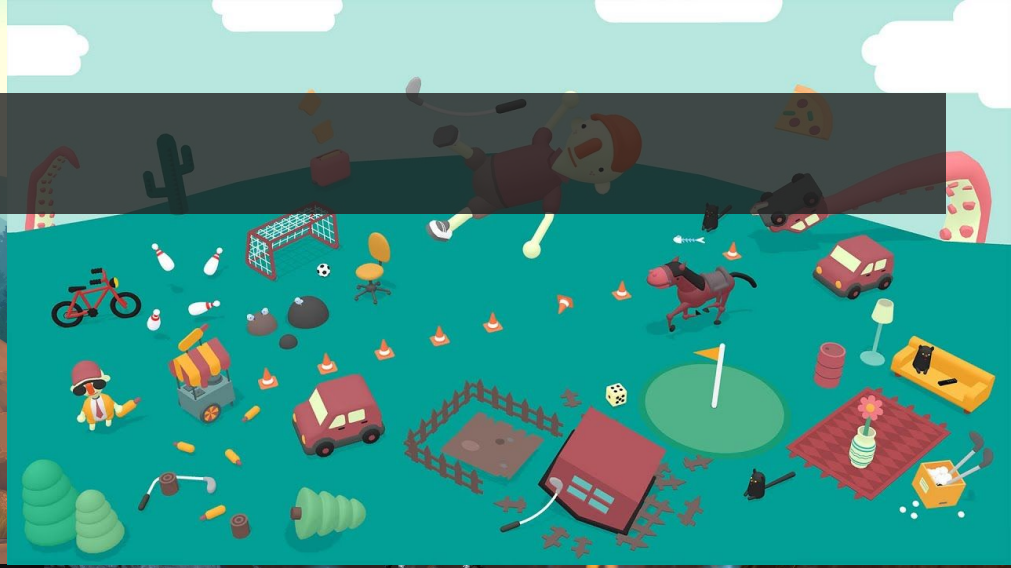
Agenda

1. Physics vs Collisions
2. Geometric overlaps and separations
3. Optimizing collision detection

Agenda

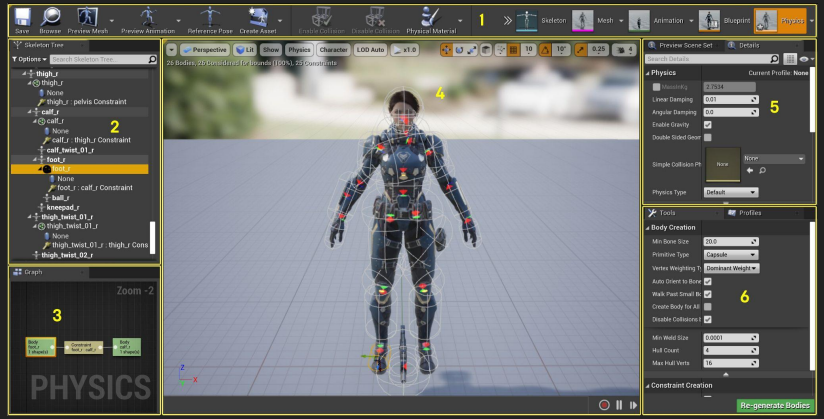
1. Physics vs Collisions
2. Geometric overlaps and separations
3. Optimizing collision detection

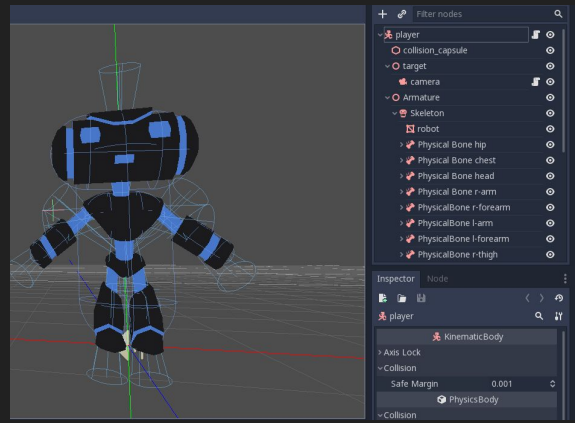
Simulation vs Control



Impact of physics on a Game

- Design
  - loss of control and predictability
  - emergent behaviours
- Engineering
  - more tools needed
  - all systems need to work with it accordingly
- Art
  - tools are more complex
  - content needs to be aware of physic
  - how does content interact with physics? (ie, animations vs physics)





3rd party Collision/Physics

- Open Dynamics Engine (ODE): Free and full source
- Bullet: Used for games and film, free and supports CCD (continuous collision detection)
- TrueAxis: free for non commercial use
- Jolt Physics: MIT licence, a few AAA games shipped with it
- PhysX: nVIDIA’s free lib (for GPU co-processing), open source since 2018
- Havok: Commercial (3D) gold standard, pay to use
- Box2D: 2D physics and collision library, free, open source

3rd party Collision/Physics

- Open Dynamics Engine (ODE): Free and full source
- Bullet: Used for games and film, free and supports CCD (continuous collision detection)
- TrueAxis: free for non commercial use
- Jolt Physics: MIT licence, a few AAA games shipped with it
- PhysX: nVIDIA's free lib (for GPU co-processing), open source since 2018
- Havok: Commercial (3D) gold standard, pay to use
- Box2D: 2D physics and collision library, free, open source

Good options for your final project if you want 3D

used in L04 Physics

Collision detection vs physics simulation.

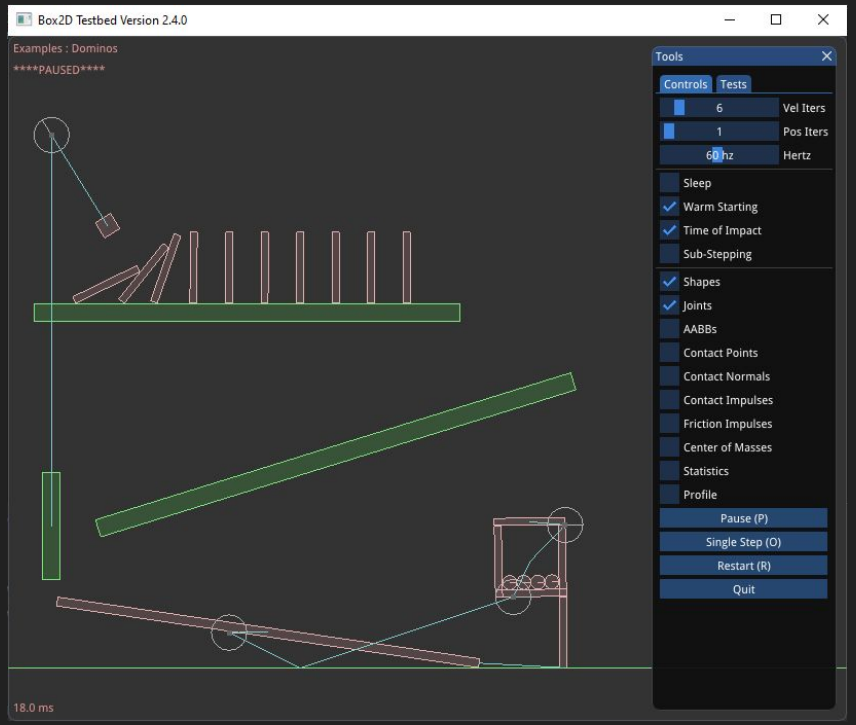
- Physics simulations depends strongly on efficient collision detection.
- APIs often have clean separation between the two concepts.

Agenda

1. Physics vs Collisions
2. Geometric overlaps and separations
3. Optimizing collision detection

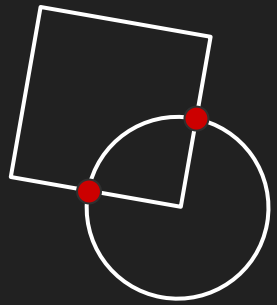
The Glorified Geometric Intersection Tester?

- The collision system detects if any pairs objects collide
- Objects are represented by simplified primitive shapes
- If a collision is detected contact information is generated, to allow object separation

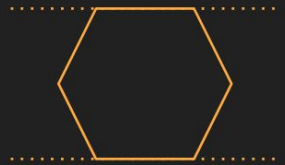


Collision Terms

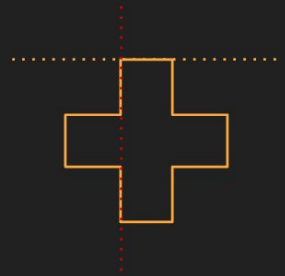
- Contact (or collision)
- Intersection (or overlap/penetration)
- Convexity (Convex vs Concave)



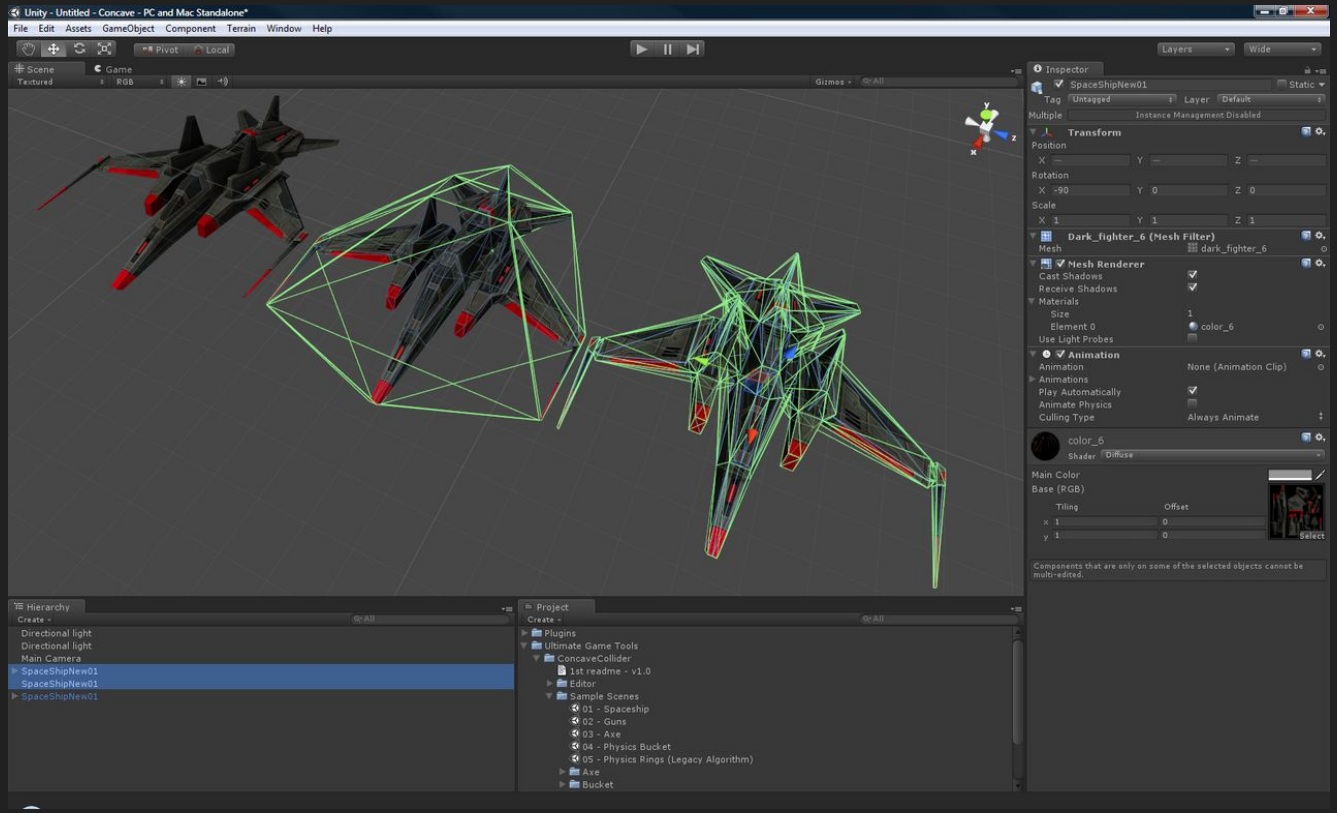
Convex polygon



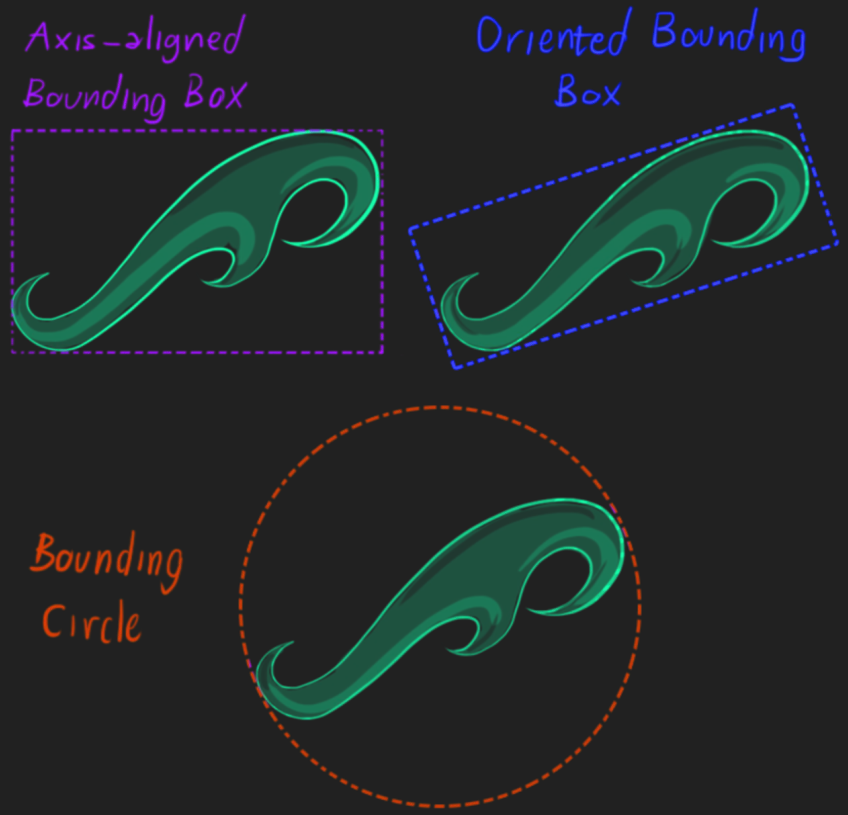
Concave polygon



Collision representations

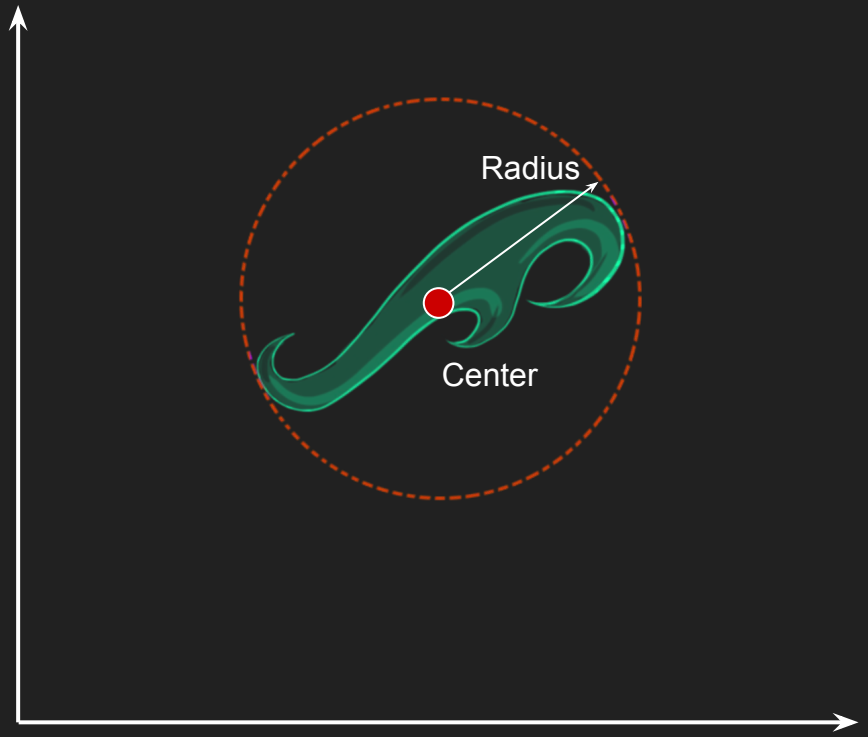


Bounding representation



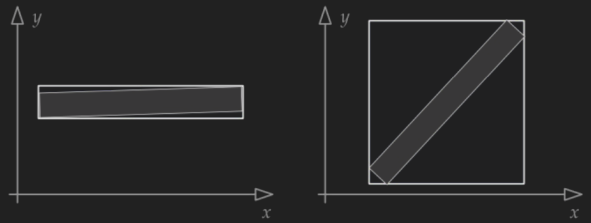
Bounding circle

- Represented by
  - Center position
  - Radius
- A point collides with bounding circle if distance to center is less than radius: `|center-p| < radius`



Axis-Aligned Bounding Box (AABB)

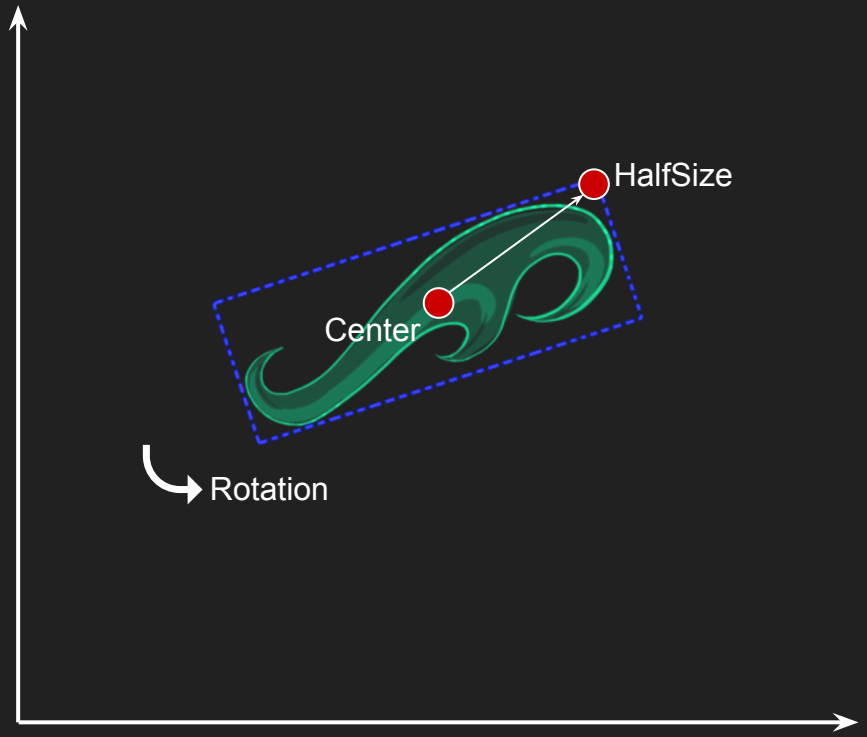
- Represented by either
  - min + max positions
  - Center position + half-size
- Has to redefined if object rotates



- A point collides with AABB if larger than min and less than max:
  min < p < max

Oriented Bounding Box (OBB)

- Represented by
  - Center position
  - half-size
  - rotation



Collision Primitives

Fastest

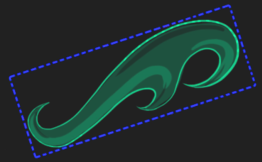
1. Spheres (or Circles in 2D)
2. Capsules
3. Axis-Aligned Bounding Boxes (AABB)
4. Oriented Bounding Boxes (OBB)
5. Discrete Oriented Polytopes (DOP)
6. Arbitrary Convex Volumes
7. Poly Soup

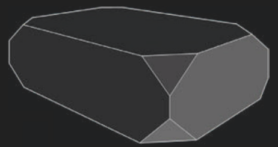
Slowest

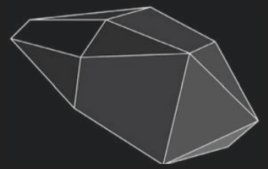
1 

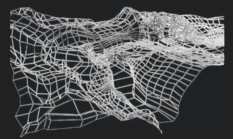
2 

3 

4 

5 

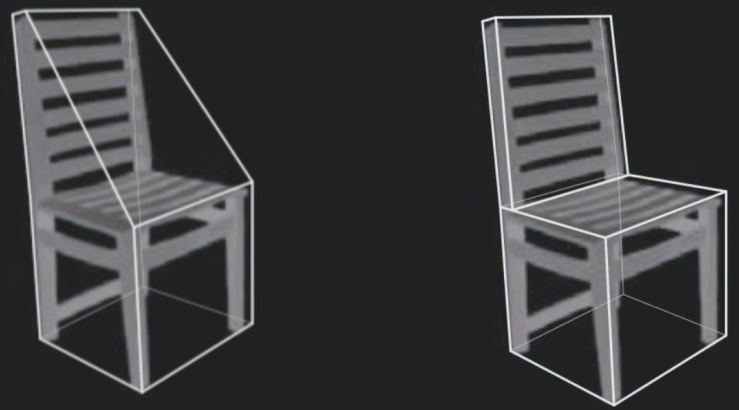
6 

7 

Compound shapes

- A concave object can be approximated by multiple convex colliders

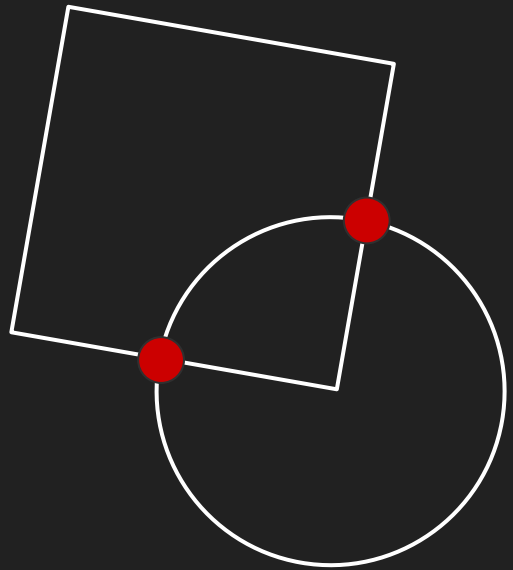
- Convex Decomposition: The process of cutting a concave object into convex parts



Collision testing

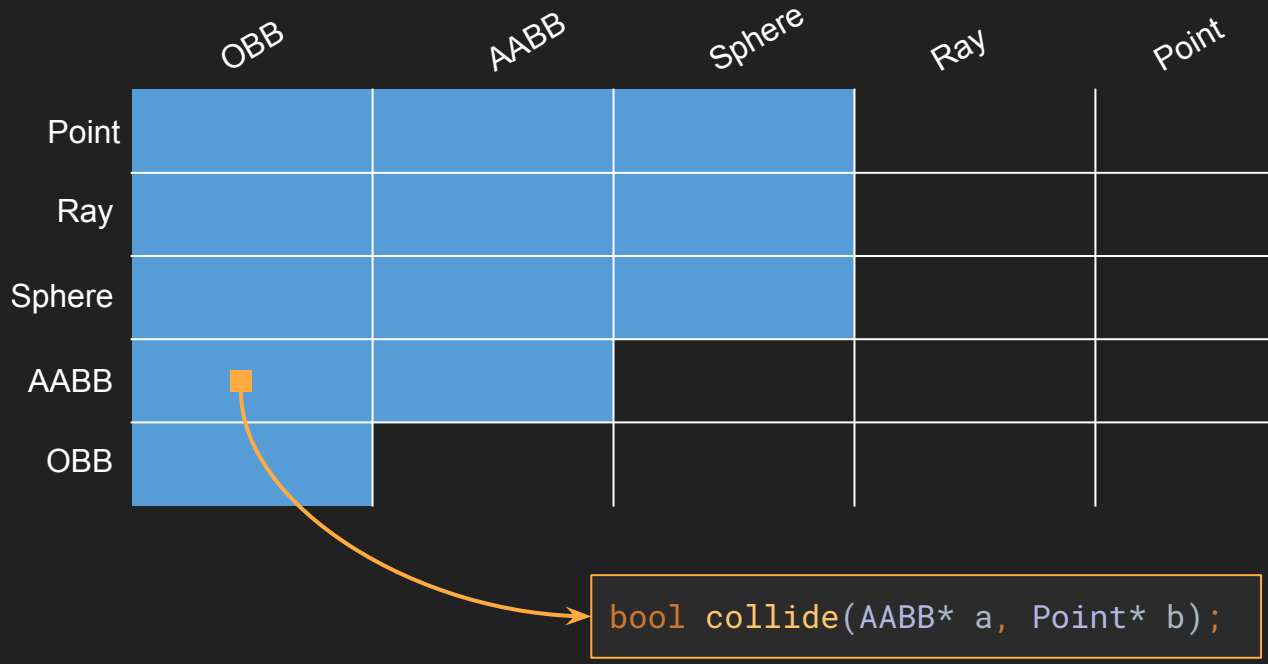
Determine if two collision primitives collide (and how)

- Point vs Sphere
- Sphere vs Sphere
- AABB vs Point
- AABB vs Sphere
- AABB vs AABB
- Etc.

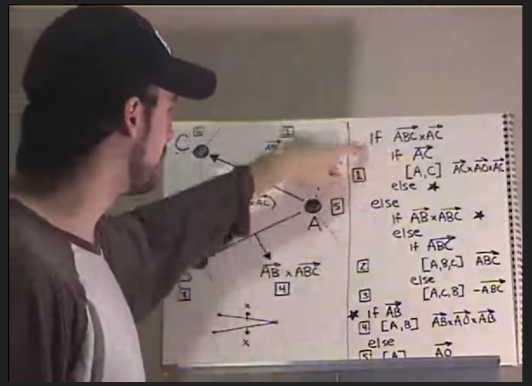


Collision testing

Physics system need to be able to test against pairs of collision primitives



GJK Algorithm



Simple but powerful algorithm to handle arbitrary convex shape.

References:

- Original paper: https://graphics.stanford.edu/courses/cs448b-00-winter/papers/gilbert.pdf
- Good intuitive explanation: https://caseymuratori.com/blog_0003
- Modern stuff (also hardcore simulations, not just games)
  - https://arxiv.org/pdf/2205.09663
  - https://graphics.stanford.edu/courses/cs468-01-fall/Papers/van-den-bergen.pdf
  - https://ora.ox.ac.uk/objects/uuid:69c743d9-73de-4aff-8e6f-b4dd7c010907

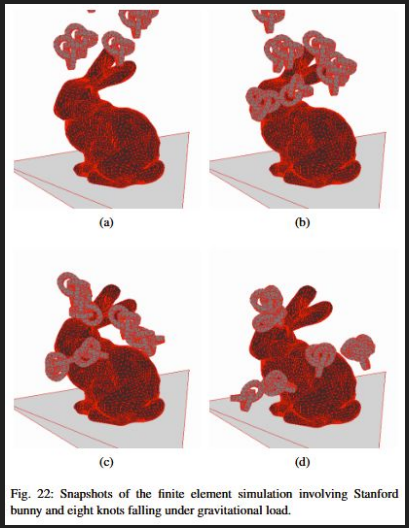
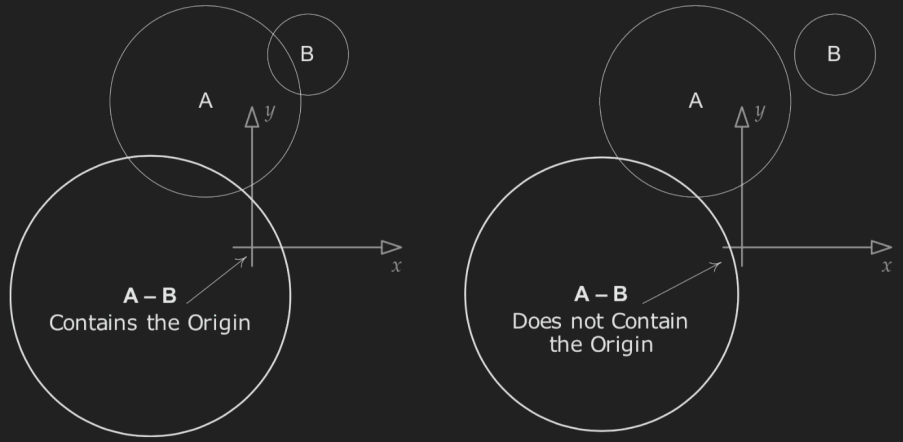


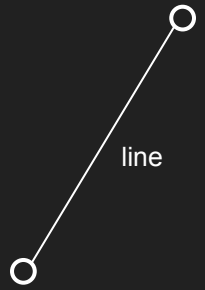
Fig. 22: Snapshots of the finite element simulation involving Stanford bunny and eight knots falling under gravitational load.

GJK Insights - Minkowsky difference

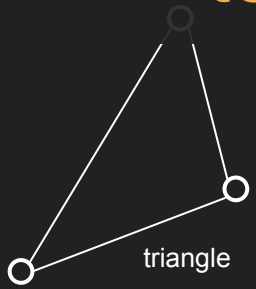
- geometrical entity constructed by taking the difference between "all points" two different shapes
- The Minkowsky difference contains the origin IIF the two original shapes are colliding
- Two main questions tho:
  - how do we do this operation efficiently?
  - how do we test if an arbitrary (convex) shape contains the origin?

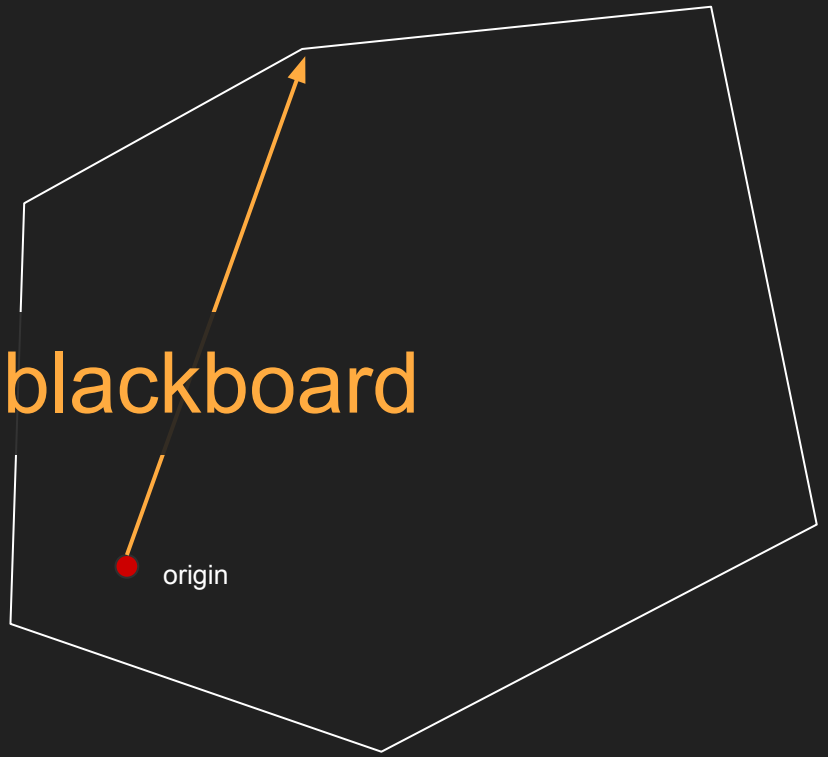


GJK Insights - support function and simplex



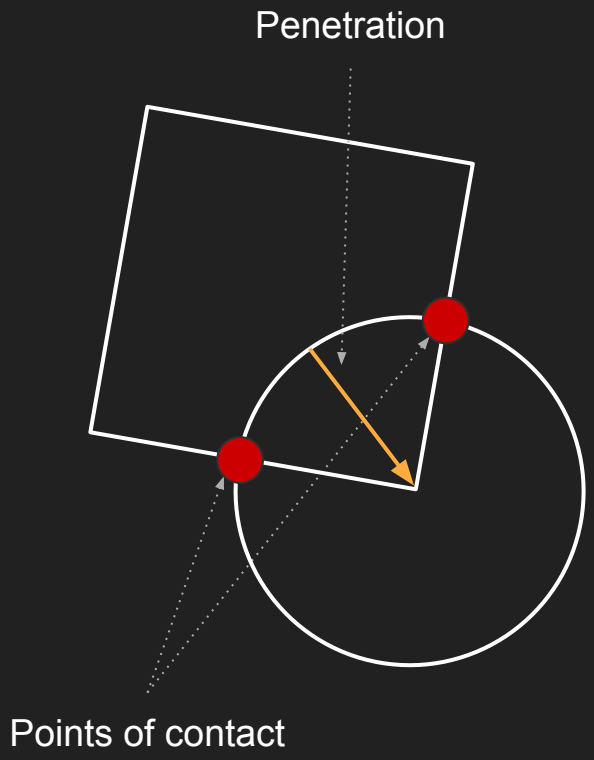
to the (virtual) blackboard





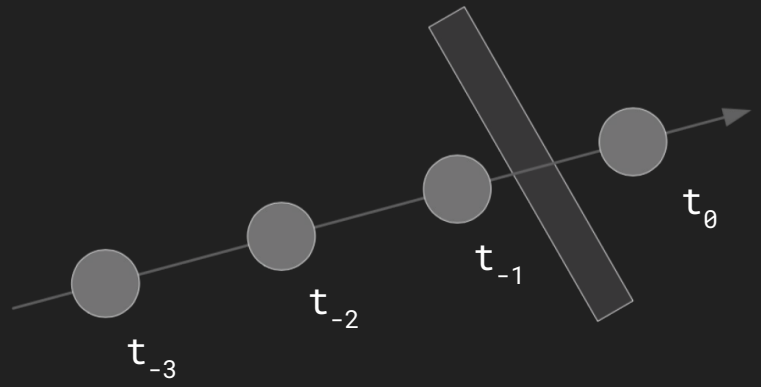
Separation

- Calculate the point of contact
- Calculate collision direction
- Calculate overlapping depth
- Use that for separation, eg. move one object back



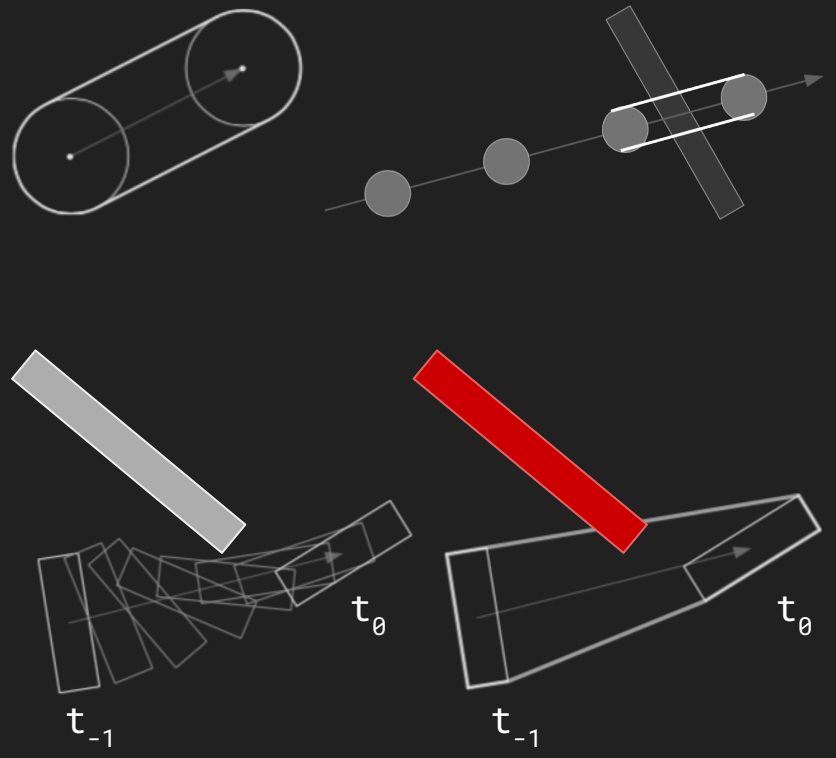
Discrete detection

- At each simulated timestep, move all objects to their new locations, then do static testing
- Works well for objects that move slowly relative to their size, and is default in most engines
- Fast movement relative to size produce gaps, which means small objects than be “overlooked”



Swept Shapes

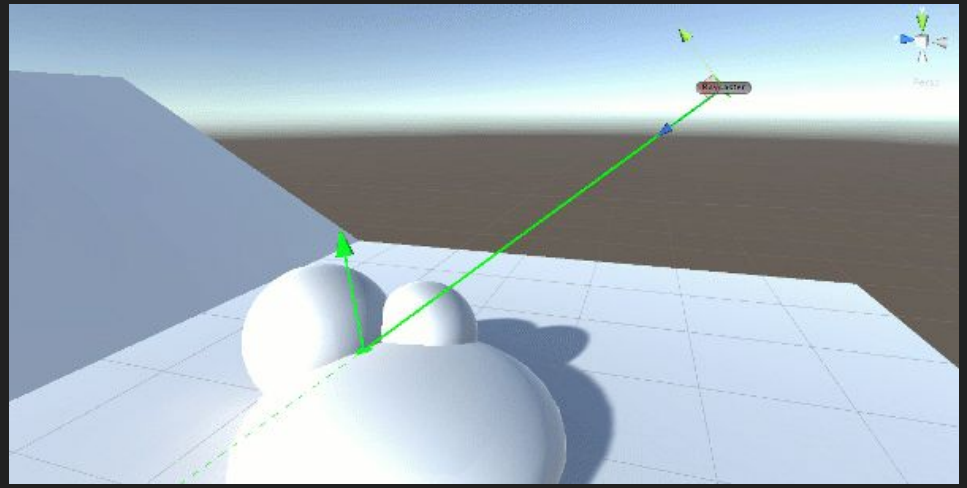
- Swept Sphere becomes a Capsule
- Then calculate static collisions on the resulting shapes
- Issues with objects that follow curved path or that are rotating, might create a concave shape.

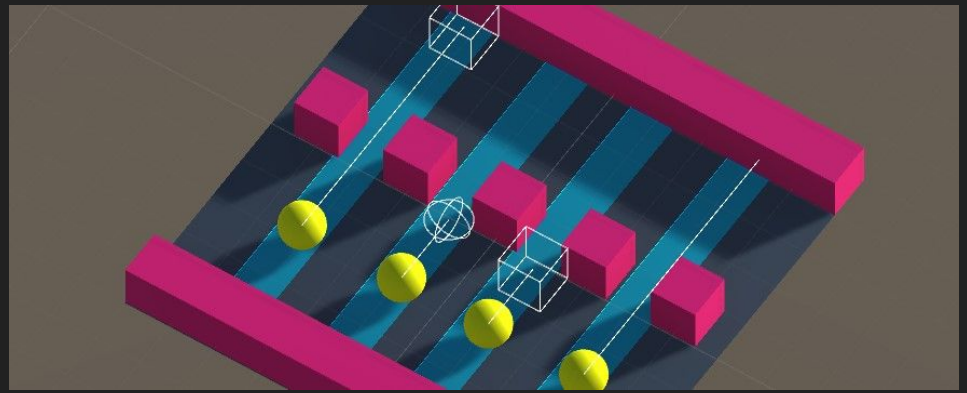


Collision Queries

Hypothetical collision tests (no impact, just info)

- Point test
- Ray casting
- Shape casting





Agenda

1. Physics vs Collisions
2. Geometric overlaps and separations
3. Optimizing collision detection

Optimizing collision detection

Finding collisions between all objects is expensive

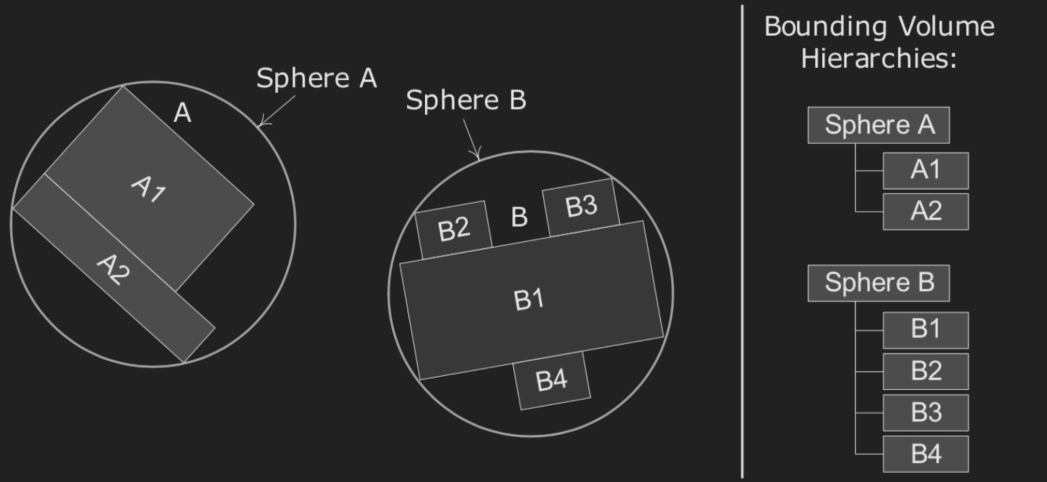
- because a single call to check collision can be expensive (see previous slides)
- because we are doing a lot of them (naive approach: $O(n^2)$)

```
for(int i = 0; i < entities_alive_count; ++i)
    for(int j = 0; j < entities_alive_count; ++j)
        if(entity_collision_check(entities[i], entities[j]))
        {
            // handle collision
        }
```

Collision detection optimizations

- Pairs optimization
  - Bounding volumes hierarchies
- Global optimization
  - Static colliders
  - Collision layers
  - World partitioning

Is it worth running the full collision test between A and B?



test complex shapes only if simple bounding sphere/boxes are colliding

Collision detection optimizations

- Pairs optimization
  - Bounding volumes hierarchies
- Global optimization
  - Static colliders
  - Collision layers
  - World partitioning

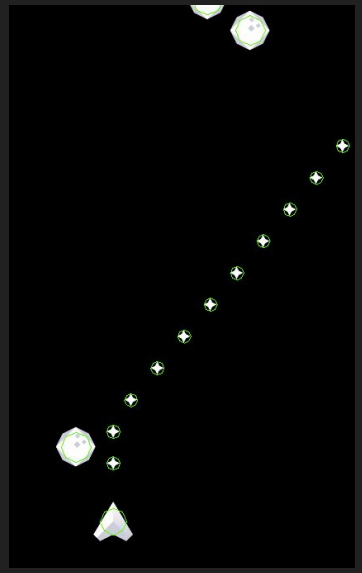
Immobile objects like platforms, ground etc will never collide, unless someone collides with them

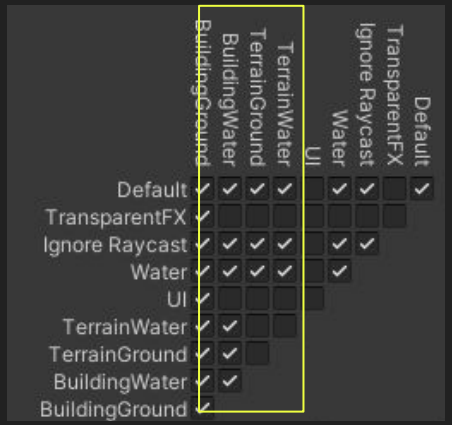
test dynamic colliders VS all colliders (instead of all VS all)

Collision detection optimizations

- Pairs optimization
  - Bounding volumes hierarchies
- Global optimization
  - Static colliders
  - Collision layers
  - World partitioning

Do we care about collisions between A and B?



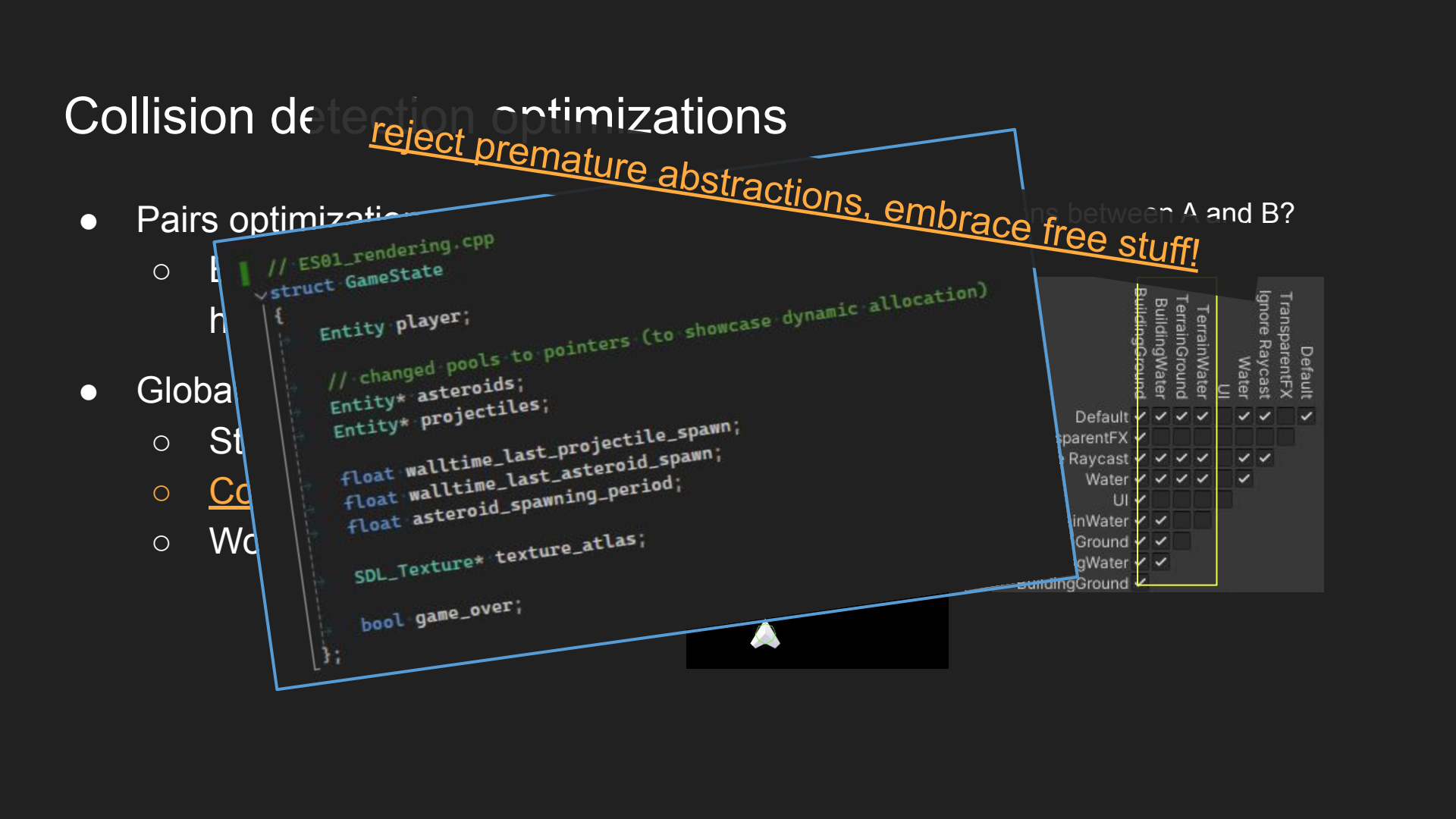


Collision detection optimizations

- Pairs optimizati
  - h
- Globa
  - St
  - Co
  - Wo

reject premature abstractions, embrace free stuff!

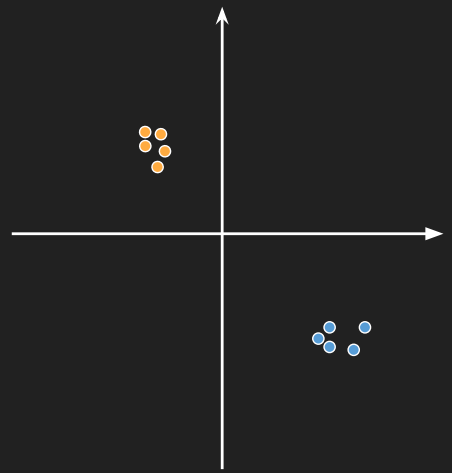
```cpp
// ES01_rendering.cpp
struct GameState
{
    Entity player;

    // changed pools to pointers (to showcase dynamic allocation)
    Entity* asteroids;
    Entity* projectiles;

    float walltime_last_projectile_spawn;
    float walltime_last_asteroid_spawn;
    float asteroid_spawning_period;

    SDL_Texture* texture_atlas;

    bool game_over;
};
```



Collision detection optimizations

- Pairs optimization
  - Bounding volumes hierarchies
- Global optimization
  - Static colliders
  - Collision layers
  - World partitioning

Is there ANY chance these objects can collide?

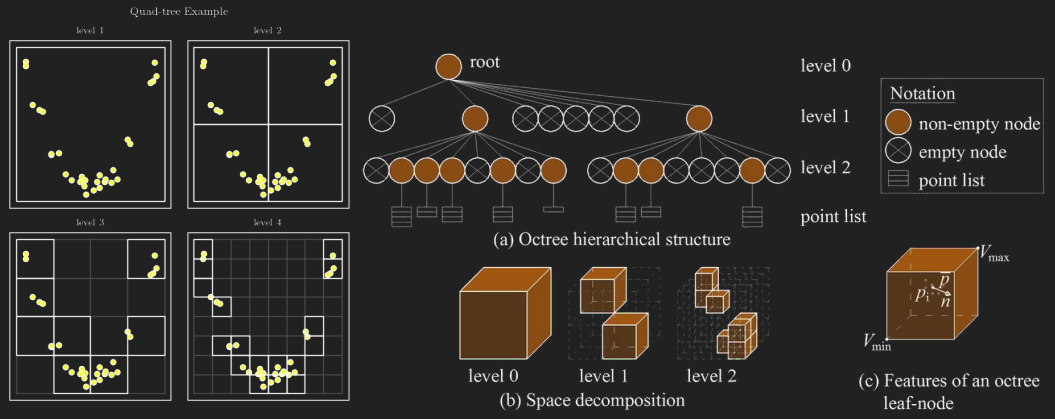


spit the world in different regions, test only objects in the same region

World Partitioning algorithms

- Quadtrees/Octrees
- Dynamic AABB Trees
- Others

- Recursively subdivide space in equal size regions.
- Regions will be lower resolutions for regions with less objects
- Perform collision test only between objects in the same region
- Remember, objects can belong to multiple regions when they cross the boundaries

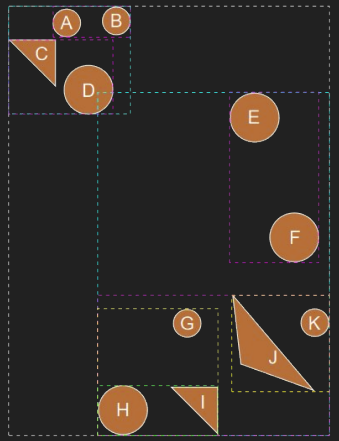


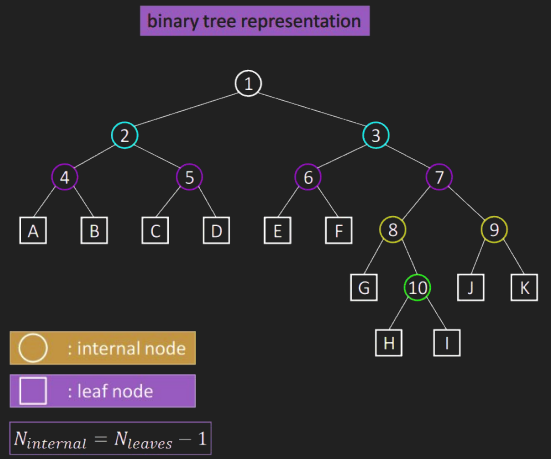
Interactive explanation: https://jimkang.com/quadtreevis/

World Partitioning algorithms

- Quadtrees/Octrees
- Dynamic AABB Trees
- Others

- Recursively group objects' AABB into pairs
- Leaves are objects' AABB, internal nodes are union of children's AABB
- If object is not colliding with a (internal) node, it can't collide with any children!
- can be extremely effective, but keeping the tree balanced is key

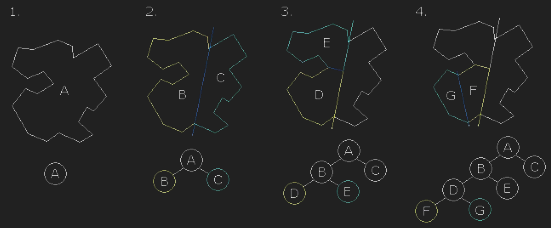




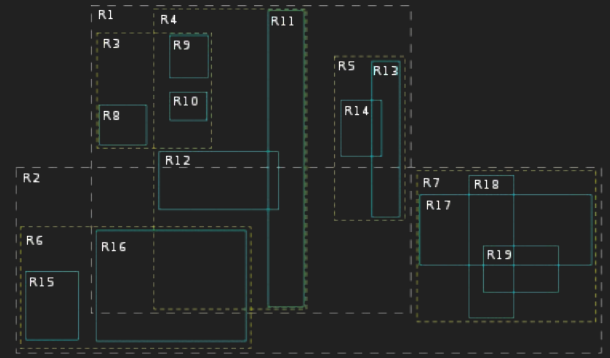
https://box2d.org/files/ErinCatto_DynamicBVH_GDC2019.pdf

World Partitioning algorithms

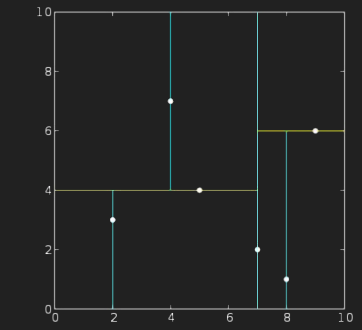
- Quadtrees/Octrees
- Dynamic AABB Trees
- Others



Binary Partition Tree (BPS)



R-Tree



k-d Tree

https://en.wikipedia.org/wiki/Space_partitioning#In_computer_graphics

Collision detection revised

Collision detection is now performed in two steps:

- Broad-phase: separate objects into multiple groups
- Narrow-phase: intersection tests on all objects within this groups

To study for the exam

- types of colliders
- generic overview of the GJK algorithm
- difference between broad and narrow phase
- ways to optimize collision detection
- overview of world partition algorithms

Today’s exercise